Collection Framework – Valid Parentheses

Observation Task: Week-14 (Experiment No: 14, CO4) Date: __ / __ / 2026

■ Objective / Aim

To write a Java program that, given a string containing just the characters '(', ')', '{', '}', '[' and ']', determines whether the input string is valid, using the Collection Framework classes LinkedList (as a stack), HashMap and HashSet. A string is valid if (1) open brackets are closed by the same type of bracket and (2) open brackets are closed in the correct order. An empty string is also valid. Examples: input ( ) gives **valid**; input ({) } gives **Not valid**.

■ System Requirements

Hardware

- Standard PC/Laptop
- Intel Core i3/i5 processor or equivalent
- Minimum 4 GB RAM
- Minimum 500 MB free storage

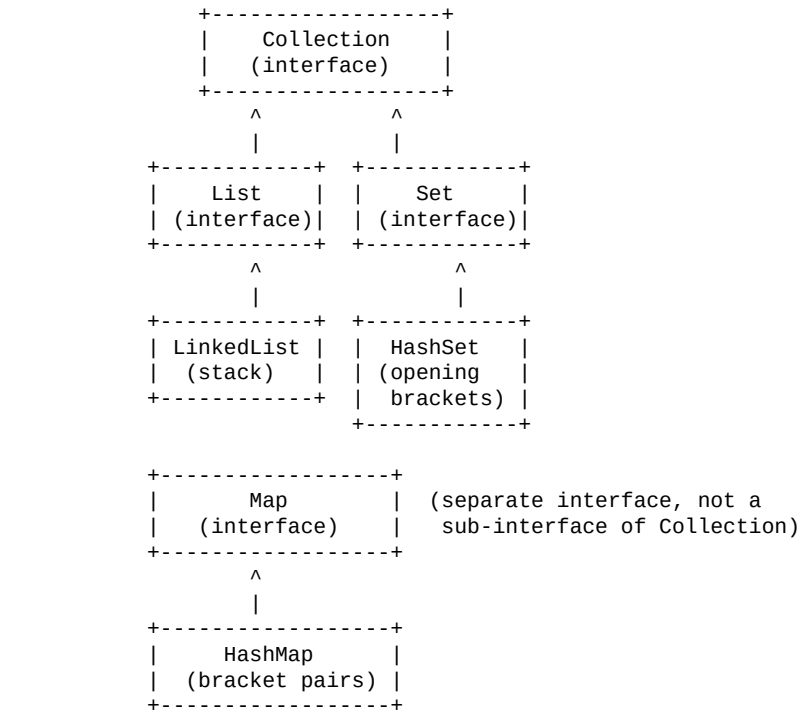
Software

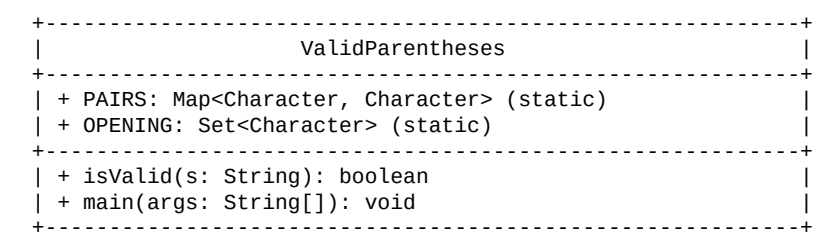
- Operating System: Windows / Linux / macOS
- Java Development Kit (JDK 17 or JDK 21)
- Text Editor or IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code
- Command Prompt / Terminal

■ Algorithm / Logic Description

1. Create a class named **ValidParentheses**.
2. Store the bracket pairs in a **Map<Character, Character>** (HashMap): closing bracket to its opening bracket, i.e.) = (, } = {,] = [.
3. Store the opening brackets in a **Set<Character>** (HashSet) created from the values of the map: ({ [.
4. Define **isValid(String s)** that returns a boolean:
   - Create a **LinkedList<Character>** to be used as a stack.
   - For each character: if it is an opening bracket (found in the Set), push it onto the stack.
   - If it is a closing bracket (key of the Map), return false when the stack is empty or the popped bracket is not its matching opening bracket.
   - If the character is not a bracket at all, return false.
   - After all characters are processed, return true only if the stack is empty (so an empty string is valid).
5. In main(), read the string using Scanner and remove the spaces from it.
6. Call isValid() and print "Output: valid" or "Output: Not valid".

■ UML Class Diagram





Relationship: LinkedList implements List (and Deque/Queue), and HashSet implements Set. List and Set are sub-interfaces of Collection, while Map is a separate interface of the framework. ValidParentheses uses (association) a HashMap for the bracket pairs, a HashSet to recognise opening brackets, and a LinkedList as the stack.

■ Source Code

```
import java.util.*;

public class ValidParentheses {

    // Map: closing bracket -> matching opening bracket
    static final Map<Character, Character> PAIRS = new HashMap<>();
    static {
        PAIRS.put(')', '(');
        PAIRS.put('}', '{');
        PAIRS.put(']', '[');
    }

    // Set: all opening brackets
    static final Set<Character> OPENING = new HashSet<>(PAIRS.values());

    static boolean isValid(String s) {
        LinkedList<Character> stack = new LinkedList<>(); // LinkedList used as a stack
        for (char ch : s.toCharArray()) {
            if (OPENING.contains(ch)) {
                stack.push(ch);                           // open bracket -> push
            } else if (PAIRS.containsKey(ch)) {
                if (stack.isEmpty() || !stack.pop().equals(PAIRS.get(ch))) {
```

```
                    return false;                           // wrong type or wrong order
                }
            } else {
                return false;                               // not a bracket character
            }
        }
        return stack.isEmpty();                             // empty stack -> all matched
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter brackets: ");
        String input = sc.nextLine().replaceAll("\\s+", ""); // ignore spaces
        System.out.println(isValid(input) ? "Output: valid" : "Output: Not valid");
        sc.close();
    }
}
```

■ Compilation & Execution Commands

Save the source code exactly as:

`ValidParentheses.java`

Compile the program:

```
javac ValidParentheses.java
```

Run the compiled program:

```
java ValidParentheses
```

■ Output Screen

Run 1

```
Enter brackets: ( )
Output: valid
```

Run 2

```
Enter brackets: ( { ) }
Output: Not valid
```

Run 3

```
Enter brackets: { [ ( ) ] }
Output: valid
```

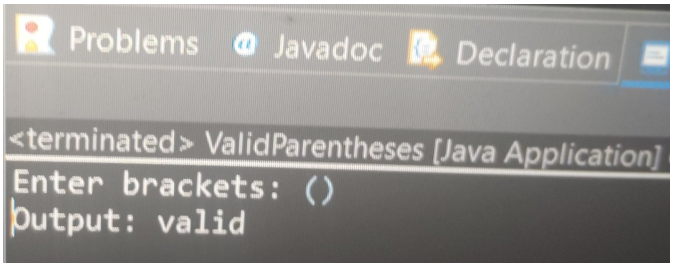
Run 4

```
Enter brackets:
Output: valid
```

Run 5

```
Enter brackets: ( ]
Output: Not valid
```

Run 6 (Screenshot)



■ Result

The program was successfully implemented in Java using the Collection Framework. The bracket pairs are stored in a Map, the opening brackets in a Set, and a LinkedList is used as a stack. For the input ( ) the program printed valid and for ({) } it printed Not valid; an empty input is treated as valid.